mra.py 中异常分数与 shift score 模块的科研式说明手册

2026-04-17

摘要

本文面向仓库当前版本的 mra.py，以科研论文说明书的形式分析异常分数计算模块，重点展开其中的 shift score。文档首先定位代码入口和训练入口，然后说明训练流程、参数更新过程、训练集估计出的参数如何作用于测试集。核心结论是：shift score 不是一个可反向传播的神经网络层，而是一个基于训练集补全窗口统计特征的分布偏移度量；它先从训练集补全窗口中估计特征均值和标准差，再用这些训练集统计量去衡量测试补全窗口离训练分布有多远。为了让数学原理易懂，本文把公式和代码逐行对应，并给出输入、输出张量形状和数据在内部流动的过程。图学习、频域插补和 Transformer 检测器只在解释 shift score 上下游依赖时简要说明，不作为本文主体。

关键词：异常检测，分布偏移，shift score，多采样率，补全窗口，标准化距离

目录

1 文档范围与核心结论 **3**
1.1 本文重点 . . . 3
1.2 先给结论 . . . 3

2 代码入口、训练入口与总体调用链 **4**
2.1 入口总表 . . . 4
2.2 两类“训练入口”必须分开 . . . 4
2.3 完整调用链 . . . 5

3 数据、窗口与张量形状 **5**
3.1 默认数据形状 . . . 5
3.2 标准化器对 shift score 的影响 . . . 5
3.3 窗口形状 . . . 6
3.4 shift score 使用的窗口不是原始评分窗口 . . . 6

4 联合模型训练流程与参数更新 **6**
4.1 训练 batch 内部的数据流 . . . 6
4.2 主要输出张量形状 . . . 7
4.3 损失函数和参数更新 . . . 7

5 shift score 的数学原理 **8**
5.1 它想解决什么问题 8
5.2 窗口特征构造 8
5.3 特征维度拆解 9
5.4 从窗口特征到分布偏移分数 10
5.5 通俗解释 10
5.6 与马氏距离的关系 11

6 shift score 的输入、输出与内部数据流 **11**
6.1 函数级输入输出 11
6.2 主流程中的实际形状 11
6.3 为什么要先补全再计算 12

7 异常分数合成与测试集作用 **12**
7.1 三类分数 12
7.2 检测器分数 12
7.3 专家分歧分数 12
7.4 shift score 的合成方式 12
7.5 平滑、阈值与最终预测 13
7.6 训练集统计量在测试集中的作用 13

8 代码实现逐段说明 **14**
8.1 `build_window_feature_matrix` 14
8.2 `compute_distribution_shift_scores` 14
8.3 `main` 中的接入位置 15

9 设计思路、优点与局限 **15**
9.1 设计思路 15
9.2 为什么和重构误差互补 15
9.3 当前实现的优点 16
9.4 当前实现的局限 16

10 常见误解与正确理解 **16**

11 可复现实验与实现建议 **16**
11.1 建议输出的诊断量 16
11.2 如果要把训练和测试拆开 17
11.3 如果要提升通用性 17

12 结论 **17**

1 文档范围与核心结论

1.1 本文重点

本文重点分析以下代码区域：

- `mra.py:740--773`：各类异常分数的训练集归一化与最终合成；
- `mra.py:815--843`：`shift score` 的特征构造与分布偏移分数计算；
- `mra.py:993--1055`：训练集和测试集统计量采集、`shift score` 接入最终异常分数、平滑、阈值化；
- `mra.py:563--636`：神经网络联合训练入口，用来说明哪些参数通过反向传播更新；
- `mra.py:954--1029`：训练后的补全序列如何被重新窗口化，并成为 `shift score` 的输入。

其它模块只在必要处说明。比如双专家插补器和 Transformer 检测器会影响 `shift score` 的输入，但它们自身的完整结构不是本文重点。

1.2 先给结论

结合代码阅读和默认数据形状探针，可以得到以下核心结论：

- `shift score` 的代码入口是 `compute_distribution_shift_scores`，位于 `mra.py:830--843`；它使用的特征入口是 `build_window_feature_matrix`，位于 `mra.py:815--827`。
- `shift score` 的主流程入口在 `main` 中，位于 `mra.py:1012--1029`。代码先把训练集和测试集的补全序列重新切成评分窗口，再计算训练窗口和测试窗口的分布偏移分数。
- `shift score` 本身没有神经网络参数，也没有 `loss.backward()` 和 `optimizer.step()`。它的“训练”是统计意义上的拟合：用训练集窗口特征估计均值 $\boldsymbol{\mu}_f$ 和标准差 $\boldsymbol{\sigma}_f$。
- 神经网络参数真正更新的位置是 `mra.py:563--636` 的 `train_model`。这里使用 AdamW 同时更新图专家、频域专家、门控网络和 Transformer 检测器。
- `shift score` 并不直接使用原始含缺失值的窗口，而是使用训练后模型生成的稠密补全序列 `train_complete_scaled` 和 `test_complete_scaled`。
- 默认数据有 18 个变量，窗口长度为 50。训练评分窗口为 $4001 \times 50 \times 18$，测试评分窗口为 $4000 \times 50 \times 18$。`shift score` 每个窗口提取 108 维统计特征。
- `shift score` 先计算每个窗口相对训练特征均值的标准化距离，再进入 `combine_scores`。在最终分数里，它与 `detector_score` 和 `disagreement_score` 合成。
- `shift score` 的训练集分数不是异常标签，而是校准基准。训练集分数用于估计该分数通道自己的均值、标准差，并参与训练分数阈值的确定。
- 当前实现中，`shift score` 的特征均值和标准差是函数内局部变量，没有保存到 `model.pt`。如果以后要把训练和测试拆成两个独立进程，需要额外保存这两个统计量。

2 代码入口、训练入口与总体调用链

2.1 入口总表

入口	代码位置	作用
命令行权重入口	`mra.py:104--108`	定义 `--score-shift-weight`，控制 `shift score` 在最终异常分数中的权重，默认值为 1.0。
训练窗口入口	`mra.py:875--898`	构造插补训练窗口、训练评分窗口和测试评分窗口。
神经网络训练入口	`mra.py:927--941` 调用 `mra.py:563--636`	训练双专家插补器和 Transformer 检测器，更新神经网络权重。
补全序列入口	`mra.py:954--973`	用训练后的模型生成 `train_complete_scaled` 和 `test_complete_scaled`。
基础统计分数入口	`mra.py:993--1010` 调用 `mra.py:680--737`	计算 `detector_score`、`disagreement_score` 和 `gate_entropy`。
`shift score` 窗口入口	`mra.py:1012--1023`	把补全后的训练集和测试集重新切成无缺失评分窗口。
`shift score` 计算入口	`mra.py:1024--1029` 调用 `mra.py:830--843`	用训练集补全窗口拟合分布统计量，并给训练/测试窗口计算偏移分数。
特征构造入口	`mra.py:815--827`	从每个窗口中提取均值、标准差、一阶变化和 FFT 幅值特征。
分数合成入口	`mra.py:1031--1036` 调用 `mra.py:752--773`	把三类分数统一到训练集尺度后合成最终原始分数。
平滑和阈值入口	`mra.py:1037--1055`	对分数做 EWAF 平滑，用训练分数确定阈值，再生成测试预测。

2.2 两类“训练入口”必须分开

这份代码里有两种容易混淆的“训练”。

第一类：神经网络训练。 这是真正的反向传播训练，入口是 `train_model`。每个 batch 中，代码随机遮挡一部分原本可观测的位置，令模型去恢复这些位置。随后计算五项损失，加权求和，执行反向传播和 AdamW 更新。对应代码为：

$$\begin{aligned}\texttt{outputs = model(...)} &\to \texttt{losses = compute_losses(...)} \\ &\to \texttt{backward()} \to \texttt{optimizer.step()}.\end{aligned} \tag{1}$$

第二类：`shift score` 的统计拟合。 `shift score` 不执行反向传播。它的拟合过程只有三步：

1. 从训练集补全窗口中提取特征矩阵 $\boldsymbol{F}^{tr}$；
2. 计算训练特征均值 $\boldsymbol{\mu}_f$ 和标准差 $\boldsymbol{\sigma}_f$；
3. 用 $\boldsymbol{\mu}_f, \boldsymbol{\sigma}_f$ 标准化训练窗口和测试窗口，并计算每个窗口的标准化距离。

因此，若把 `shift score` 称为“训练过的模块”，这里的“训练”应理解为“用训练集估计统计基准”，而不是“用梯度下降学习参数”。

2.3 完整调用链

从原始数据到最终测试预测，和 `shift score` 相关的实际链路如下：

$$\begin{aligned}&\text{读取 CSV} \to \text{观测值标准化} \to \text{构造训练插补窗口} \to \text{训练联合模型}\\&\to \text{补全训练/测试全序列} \to \text{把补全序列切成评分窗口} \to \text{提取窗口特征}\\&\to \text{用训练特征拟合均值和标准差} \to \text{计算训练/测试 shift score} \to \text{合成最终异常分数.}\end{aligned} \tag{2}$$

这条链路说明一个关键事实：`shift score` 虽然没有梯度训练，但它依赖前面已经训练好的插补模型。插补模型变了，补全序列会变，窗口特征会变，`shift score` 也会随之改变。

3 数据、窗口与张量形状

3.1 默认数据形状

默认命令读取：

- `data/train/train_1.csv`；
- `data/test/test_C5_1.csv`。

当前仓库数据的实际形状为：

$$\boldsymbol{X}_{raw}^{tr} \in \mathbb{R}^{4100\times 18}, \tag{3}$$

$$\boldsymbol{X}_{raw}^{te} \in \mathbb{R}^{4100\times 18}, \tag{4}$$

$$\boldsymbol{M}_s^{tr}, \boldsymbol{M}_s^{te} \in \{0,1\}^{4100\times 18}. \tag{5}$$

其中 $\boldsymbol{M}_s$ 是结构性缺失掩码。若 $M_{s,t,n}=1$，表示第 t 个时间点、第 n 个变量在原始 CSV 中是 `NaN`。

3.2 标准化器对 `shift score` 的影响

`ObservedStandardScaler` 在 `mra.py:865--867` 被调用。它只用训练集观测值拟合每个变量的均值和标准差：

$$\mu_n = \frac{1}{|\Omega_n^{tr}|}\sum_{t\in\Omega_n^{tr}} X_{raw,t,n}^{tr}, \qquad \sigma_n = \max\left(\sqrt{\frac{1}{|\Omega_n^{tr}|}\sum_{t\in\Omega_n^{tr}}(X_{raw,t,n}^{tr}-\mu_n)^2}, 10^{-6}\right), \tag{6}$$

其中 Ω_n^{tr} 表示训练集中第 n 个变量的可观测位置集合。

训练集和测试集随后都使用同一组 μ_n, σ_n 变换：

$$X_{t,n} = \begin{cases} \dfrac{X_{raw,t,n}-\mu_n}{\sigma_n}, & M_{s,t,n}=0,\\ 0, & M_{s,t,n}=1. \end{cases} \tag{7}$$

这对 `shift score` 很重要，因为 `shift score` 使用的是 `train_complete_scaled` 和 `test_complete_scaled`。也就是说，分布偏移是在训练集标准化坐标系里衡量的。

3.3 窗口形状

默认参数为 `seq-len` $= 50$、`stride` $= 1$。代码会构造两类窗口。

插补窗口。 插补窗口由 `build_windows` 构造，用于神经网络训练和全序列补全。它允许在序列开头重复首行补齐，因此窗口数量等于原始时间步数：

$$\boldsymbol{W}_{imp}^{tr} \in \mathbb{R}^{4100\times 50\times 18}, \tag{8}$$

$$\boldsymbol{W}_{imp}^{te} \in \mathbb{R}^{4100\times 50\times 18}. \tag{9}$$

评分窗口。 评分窗口由 `build_standard_windows` 和 `build_prompt_test_windows` 构造，用于计算训练分数和测试分数。默认数据下：

$$\boldsymbol{W}_{eval}^{tr} \in \mathbb{R}^{4001\times 50\times 18}, \tag{10}$$

$$\boldsymbol{W}_{eval}^{te} \in \mathbb{R}^{4000\times 50\times 18}. \tag{11}$$

测试标签由 `build_prompt_test_windows` 生成，共 4000 个窗口，前 2000 个为正常，后 2000 个为异常。

3.4 shift score 使用的窗口不是原始评分窗口

`detector_score` 和 `disagreement_score` 使用的是 $\boldsymbol{W}_{eval}^{tr}$ 与 $\boldsymbol{W}_{eval}^{te}$。但 `shift score` 另走一条路径：

1. 先用训练后的模型生成补全序列：

$$\boldsymbol{C}^{tr}, \boldsymbol{C}^{te} \in \mathbb{R}^{4100\times 18}. \tag{12}$$

2. 再把补全序列重新切成窗口：

$$\boldsymbol{V}^{tr} = \mathrm{Window}(\boldsymbol{C}^{tr}) \in \mathbb{R}^{4001\times 50\times 18}, \tag{13}$$

$$\boldsymbol{V}^{te} = \mathrm{Window}(\boldsymbol{C}^{te}) \in \mathbb{R}^{4000\times 50\times 18}. \tag{14}$$

3. 切窗口时传入的掩码是全零矩阵，即代码中的 `np.zeros_like(train_mask)` 和 `np.zeros_like(test_mask)`。

这说明 `shift score` 衡量的是“模型补全后的稠密窗口是否像训练集补全窗口”，而不是“原始缺失模式是否像训练集”。这样做可以避免多采样率缺失模式直接主导分数，把注意力放在补全后的数据分布上。

4 联合模型训练流程与参数更新

4.1 训练 batch 内部的数据流

在 `train_model` 中，一个训练 batch 的张量记为：

$$\boldsymbol{X}_{true} \in \mathbb{R}^{B\times T\times N}, \qquad \boldsymbol{M}_s \in \{0,1\}^{B\times T\times N}. \tag{15}$$

默认 $T = 50$、$N = 18$，B 由 `batch-size` 决定，默认最多为 64。

训练时先从真实观测位置随机抽出一部分作为自监督目标：

$$\boldsymbol{M}_h = \mathrm{SampleHoldout}(\boldsymbol{M}_s), \qquad \boldsymbol{M} = \max(\boldsymbol{M}_s, \boldsymbol{M}_h). \tag{16}$$

其中 $\boldsymbol{M}_h$ 是训练期额外遮挡掩码，$\boldsymbol{M}$ 是模型实际看到的缺失掩码。

输入窗口为：

$$\boldsymbol{X}_{input} = \boldsymbol{X}_{true} \odot (1 - \boldsymbol{M}). \tag{17}$$

随后模型内部执行：

$$\boldsymbol{X}_{seed} = \mathrm{PreImpute}(\boldsymbol{X}_{input}, \boldsymbol{M}), \tag{18}$$

$$\boldsymbol{X}_{gcn} = f_{gcn}(\boldsymbol{X}_{seed}, \boldsymbol{M}), \tag{19}$$

$$\boldsymbol{X}_{freq} = f_{freq}(\boldsymbol{X}_{seed}), \tag{20}$$

$$\boldsymbol{\Pi} = \mathrm{softmax}(g(\boldsymbol{X}_{seed}, \boldsymbol{M}, \boldsymbol{X}_{gcn}, \boldsymbol{X}_{freq}, \boldsymbol{rate_id})), \tag{21}$$

$$\boldsymbol{X}_{imp} = \Pi_0 \odot \boldsymbol{X}_{gcn} + \Pi_1 \odot \boldsymbol{X}_{freq}, \tag{22}$$

$$\boldsymbol{X}_{complete} = \boldsymbol{M} \odot \boldsymbol{X}_{imp} + (1 - \boldsymbol{M}) \odot \boldsymbol{X}_{input}, \tag{23}$$

$$\widehat{\boldsymbol{X}}_{det} = f_{det}(\boldsymbol{X}_{complete}, 1 - \boldsymbol{M}_s, \boldsymbol{rate_id}, \boldsymbol{stride}). \tag{24}$$

4.2 主要输出张量形状

张量	形状	含义
$\boldsymbol{X}_{true}$	$B \times T \times N$	训练 batch 的标准化真实窗口。
$\boldsymbol{M}_s$	$B \times T \times N$	原始结构性缺失掩码。
$\boldsymbol{M}_h$	$B \times T \times N$	训练期随机遮挡出的 holdout 位置。
$\boldsymbol{M}$	$B \times T \times N$	模型实际看到的缺失掩码，等于结构性缺失和 holdout 的并集。
$\boldsymbol{X}_{seed}$	$B \times T \times N$	双向填充后的时域种子。
$\boldsymbol{X}_{gcn}$	$B \times T \times N$	图专家重构结果。
$\boldsymbol{X}_{freq}$	$B \times T \times N$	频域专家重构结果。
$\boldsymbol{\Pi}$	$B \times T \times N \times 2$	每个位置对 GCN 专家和频域专家的门控权重。
$\boldsymbol{A}$	$B \times N \times N$	图专家生成的窗口级邻接矩阵。
$\boldsymbol{X}_{complete}$	$B \times T \times N$	观测位置保留原值、缺失位置采用门控补全后的完整窗口。
$\widehat{\boldsymbol{X}}_{det}$	$B \times T \times N$	Transformer 检测器对完整窗口的重构。

4.3 损失函数和参数更新

`compute_losses` 定义五项损失：

$$\mathcal{L}_{fusion} = \frac{\sum((\boldsymbol{X}_{imp} - \boldsymbol{X}_{true}) \odot \boldsymbol{M}_h)^2}{\sum \boldsymbol{M}_h}, \tag{25}$$

$$\mathcal{L}_{gcn} = \frac{\sum((\boldsymbol{X}_{gcn} - \boldsymbol{X}_{true}) \odot \boldsymbol{M}_h)^2}{\sum \boldsymbol{M}_h}, \tag{26}$$

$$\mathcal{L}_{freq} = \frac{\sum((\boldsymbol{X}_{freq} - \boldsymbol{X}_{true}) \odot \boldsymbol{M}_h)^2}{\sum \boldsymbol{M}_h}, \tag{27}$$

$$\mathcal{L}_{graph} = \text{mean}\left((\text{diag}(\boldsymbol{A}) - \tau)^2\right), \tag{28}$$

$$\mathcal{L}_{det} = \frac{\sum((\widehat{\boldsymbol{X}}_{det} - \text{sg}(\boldsymbol{X}_{complete})) \odot (1 - \boldsymbol{M}_s))^2}{\sum(1 - \boldsymbol{M}_s)}. \tag{29}$$

其中 $\text{sg}(\cdot)$ 表示 `detach()`。它让检测器目标值不再反向传播，但 $\boldsymbol{X}_{complete}$ 作为检测器输入仍然会影响 $\widehat{\boldsymbol{X}}_{det}$，所以 $\mathcal{L}_{det}$ 仍可能反向更新插补器。

总损失为：

$$\mathcal{L} = 1.0\mathcal{L}_{fusion} + 0.4\mathcal{L}_{gcn} + 0.4\mathcal{L}_{freq} + 0.5\mathcal{L}_{det} + 0.1\mathcal{L}_{graph}. \tag{30}$$

参数更新流程为：

1. `optimizer.zero_grad()` 清空旧梯度；
2. `losses["total_loss"].backward()` 计算梯度；
3. `clip_grad_norm_(model.parameters(), max_norm=1.0)` 做梯度裁剪；
4. `optimizer.step()` 用 AdamW 更新神经网络参数。

`shift score` 不在这四步里。它没有梯度，也不会改变神经网络权重；它只在训练结束后读取补全结果并估计统计基准。

5 shift score 的数学原理

5.1 它想解决什么问题

`detector_score` 衡量的是检测器重构误差，`disagreement_score` 衡量的是两个插补专家之间的分歧。这两类分数都很有用，但仍有一个缺口：如果测试窗口整体分布已经偏离训练集，但模型仍能给出平滑补全，重构误差和专家分歧未必会很大。

`shift score` 用来补这个缺口。它不问“模型重构错了多少”，而问：

> 这个补全后的窗口，在均值、波动、变化速度和频域能量上，还像不像训练集窗口？

如果一个窗口在这些统计特征上离训练集常态很远，它的 `shift score` 就会变大。

5.2 窗口特征构造

设一个补全窗口为：

$$\boldsymbol{V}_i \in \mathbb{R}^{T \times N}, \tag{31}$$

其中 i 表示第 i 个窗口。默认 $T = 50$、$N = 18$。

`build_window_feature_matrix` 提取四类特征。

第一类：变量组均值。 代码固定使用四个列切片：

$$G_1 = \{1, \ldots, 6\}, \quad G_2 = \{7, \ldots, 12\}, \quad G_3 = \{13, \ldots, 18\}, \quad G_4 = \{1, \ldots, 18\}. \tag{32}$$

对每个组内变量，计算窗口时间均值：

$$m_{i,n} = \frac{1}{T} \sum_{t=1}^{T} V_{i,t,n}. \tag{33}$$

这类特征描述窗口整体水平是否偏高或偏低。

第二类：变量组标准差。 对同样四个列切片，计算时间标准差：

$$s_{i,n} = \sqrt{\frac{1}{T}\sum_{t=1}^{T}(V_{i,t,n} - m_{i,n})^2}. \tag{34}$$

这类特征描述窗口内部波动是否变强或变弱。

第三类：一阶变化幅度。 代码计算相邻时间点差分的平均绝对值：

$$d_{i,n} = \frac{1}{T-1}\sum_{t=2}^{T}|V_{i,t,n} - V_{i,t-1,n}|. \tag{35}$$

它描述曲线在窗口内变化得快不快。如果某个变量突然抖动、跳变或振荡增强，这个特征会变大。

第四类：频域幅值。 代码沿时间维做实数快速傅里叶变换：

$$\widehat{V}_{i,k,n} = \sum_{t=0}^{T-1} V_{i,t,n}\exp\left(-\frac{2\pi \mathrm{i}kt}{T}\right). \tag{36}$$

随后取第 1 到第 5 个非零频点的幅值均值：

$$a_{i,n} = \frac{1}{5}\sum_{k=1}^{5}|\widehat{V}_{i,k,n}|. \tag{37}$$

这里跳过第 0 个频点，是因为第 0 个频点主要表示窗口平均水平，而平均水平已经由前面的均值特征表达。非零频点更能表达周期性和振荡强度。

5.3 特征维度拆解

默认 $N = 18$ 时，每个窗口得到 108 维特征。拆解如下。

特征来源	代码对应	维度	通俗含义
第 1–6 列均值和标准差	`slice(0, 6)`	$6+6$	高频变量的水平和波动。
第 7–12 列均值和标准差	`slice(6, 12)`	$6+6$	中频变量的水平和波动。
第 13–18 列均值和标准差	`slice(12, 18)`	$6+6$	低频变量的水平和波动。
第 1–18 列均值和标准差	`slice(0, 18)`	$18+18$	全变量整体水平和波动。
相邻时间差分平均绝对值	`abs(diff(windows)).mean(axis=1)`	18	曲线变化快慢。
FFT 第 1–5 个非零频点平均幅值	`abs(rfft(windows))[:, 1:6, :].mean(axis=1)`	18	低频到中低频振荡强度。
合计	–	108	每个窗口的统计画像。

注意，这里的前三个变量组和总变量组是按当前 18 列数据写死的。它体现了当前数据的三类采样率结构：前 6 列高频，中间 6 列中频，最后 6 列低频。若未来数据列数或分组方式变化，这部分代码需要同步改造。

5.4 从窗口特征到分布偏移分数

设训练补全窗口数为 K_{tr}，测试补全窗口数为 K_{te}。默认情况下：

$$K_{tr} = 4001, \qquad K_{te} = 4000. \tag{38}$$

特征矩阵为：

$$\boldsymbol{F}^{tr} \in \mathbb{R}^{K_{tr} \times P}, \qquad \boldsymbol{F}^{te} \in \mathbb{R}^{K_{te} \times P}, \qquad P = 108. \tag{39}$$

`compute_distribution_shift_scores` 先用训练特征估计每一维特征的均值和标准差：

$$\mu_{f,j} = \frac{1}{K_{tr}} \sum_{i=1}^{K_{tr}} F_{i,j}^{tr}, \tag{40}$$

$$\sigma_{f,j} = \max\left(\sqrt{\frac{1}{K_{tr}} \sum_{i=1}^{K_{tr}} (F_{i,j}^{tr} - \mu_{f,j})^2}, 10^{-6} \right). \tag{41}$$

然后训练窗口和测试窗口都用同一组训练统计量做标准化：

$$Z_{i,j}^{tr} = \frac{F_{i,j}^{tr} - \mu_{f,j}}{\sigma_{f,j}}, \tag{42}$$

$$Z_{i,j}^{te} = \frac{F_{i,j}^{te} - \mu_{f,j}}{\sigma_{f,j}}. \tag{43}$$

最后对每个窗口计算所有特征维度的均方根距离：

$$q_i^{tr} = \sqrt{\frac{1}{P} \sum_{j=1}^{P} (Z_{i,j}^{tr})^2}, \tag{44}$$

$$q_i^{te} = \sqrt{\frac{1}{P} \sum_{j=1}^{P} (Z_{i,j}^{te})^2}. \tag{45}$$

这里的 q_i 就是原始 `shift_score`。

5.5 通俗解释

可以把每个窗口想成一张“体检表”。这张表有 108 个指标，包括平均值、波动大小、变化速度和频域振荡强度。训练集给出了正常情况下每个指标的大致范围。测试时，如果某个窗口的很多指标都偏离训练集正常范围，那么它的 `shift score` 会升高。

公式中的 $Z_{i,j}$ 是常见的 z 分数。它表示第 i 个窗口第 j 个特征离训练均值有几个训练标准差。例如：

- $Z_{i,j} = 0$：这个特征刚好等于训练平均水平；
- $Z_{i,j} = 1$：这个特征比训练均值高一个训练标准差；
- $Z_{i,j} = -2$：这个特征比训练均值低两个训练标准差。

`shift score` 对所有 $Z_{i,j}$ 平方后取平均，再开方。因此它不关心偏高还是偏低，只关心偏离程度。一个窗口只要在很多特征上都偏离训练集，就会得到较大的分数。

5.6 与马氏距离的关系

`shift score` 可以看成一种简化的标准化距离。它和马氏距离的共同点是：都先用训练集估计正常分布，再衡量新样本离正常分布有多远。

差别是：当前实现只除以每一维自己的标准差，没有使用完整协方差矩阵。也就是说，它默认这些统计特征之间相互独立，或者至少不显式建模特征间相关性。因此它更简单、更稳定，也更不容易在样本数有限时出现协方差矩阵求逆问题；代价是不能表达“两个特征单独看正常，但组合起来不正常”的情况。

6 shift score 的输入、输出与内部数据流

6.1 函数级输入输出

函数	输入/输出形状	说明
`build_window feature_matrix`	输入：$K \times T \times N$。输出：$K \times P$。默认 $P = 108$。	每个窗口变成一行统计特征。时间维被压缩掉。
`compute_distribution shift_scores`	输入：$\boldsymbol{V}^{tr} \in \mathbb{R}^{K_{tr} \times T \times N}$，$\boldsymbol{V}^{eval} \in \mathbb{R}^{K_{eval} \times T \times N}$。输出：$\boldsymbol{q}^{tr} \in \mathbb{R}^{K_{tr}}$，$\boldsymbol{q}^{eval} \in \mathbb{R}^{K_{eval}}$。	用训练窗口拟合特征均值和标准差，再同时给训练集和评估集打分。
`normalize scores`	输入：两个一维分数数组。输出：两个同形状 z 分数数组。	再次用训练分数均值和标准差统一分数通道尺度。
`combine scores`	输入：包含三类分数的一维数组字典。输出：训练原始总分和评估原始总分。	把检测器误差、专家分歧和分布偏移合成。

6.2 主流程中的实际形状

默认数据和默认窗口参数下，`shift score` 的实际形状如下：

$$\texttt{train_complete_scaled}: \quad 4100 \times 18, \tag{46}$$

$$\texttt{test_complete_scaled}: \quad 4100 \times 18, \tag{47}$$

$$\texttt{train_complete_eval_windows}: \quad 4001 \times 50 \times 18, \tag{48}$$

$$\texttt{test_complete_eval_windows}: \quad 4000 \times 50 \times 18, \tag{49}$$

$$\texttt{train_features}: \quad 4001 \times 108, \tag{50}$$

$$\texttt{test_features}: \quad 4000 \times 108, \tag{51}$$

$$\texttt{feature_mean}, \texttt{feature_std}: \quad 108, \tag{52}$$

$$\texttt{train_shift_scores}: \quad 4001, \tag{53}$$

$$\texttt{test_shift_scores}: \quad 4000. \tag{54}$$

6.3 为什么要先补全再计算

原始多采样率数据中有大量 `NaN`。如果直接对原始窗口提取均值、标准差和 FFT 特征，会遇到两个问题：

1. `NaN` 无法直接参与普通的 `mean/std/fft` 计算；
2. 如果简单把缺失值填零，分数可能主要反映采样率缺失模式，而不是真实数据分布变化。

当前实现先用训练好的双专家模型补全序列，再计算 `shift score`。这样得到的分数更接近“补全后的过程状态是否偏离训练集”，而不是“缺失模式是否偏离训练集”。

7 异常分数合成与测试集作用

7.1 三类分数

`mra.py` 中用于最终合成的三类分数为：

1. `detector_score`：检测器重构误差；
2. `disagreement_score`：GCN 专家和频域专家输出的平均绝对差；
3. `shift_score`：补全窗口相对训练补全窗口的分布偏移距离。

`gate_entropy` 也会被计算并写入 `test_predictions.csv`，但当前代码没有把它并入最终异常分数。

7.2 检测器分数

在 `collect_window_statistics` 中，检测器分数为：

$$r_i = \frac{\sum_{t,n} \left((\widehat{X}_{det,i,t,n} - X_{complete,i,t,n}) O_{i,t,n} \right)^2}{\sum_{t,n} O_{i,t,n}}, \tag{55}$$

其中 $O = 1 - M_s$ 是结构性观测掩码。它只在原始可观测位置上计算误差。

7.3 专家分歧分数

专家分歧分数为：

$$g_i = \frac{1}{TN} \sum_{t,n} |X_{gcn,i,t,n} - X_{freq,i,t,n}|. \tag{56}$$

它表示两个插补专家对同一个窗口的看法是否一致。如果两个专家差距很大，说明模型对这个窗口不够确定。

7.4 shift score 的合成方式

进入 `combine_scores` 后，三个分数通道都会用训练集的通道均值和标准差做归一化：

$$\widetilde{r}_i = \frac{r_i - \mu_r^{tr}}{\sigma_r^{tr}}, \tag{57}$$

$$\widetilde{g}_i = \frac{g_i - \mu_g^{tr}}{\sigma_g^{tr}}, \tag{58}$$

$$\widetilde{q}_i = \frac{q_i - \mu_q^{tr}}{\sigma_q^{tr}}. \tag{59}$$

最终原始异常分数为：

$$S_i = |\widetilde{r}_i| + \lambda_g \widetilde{g}_i + \lambda_q \widetilde{q}_i. \tag{60}$$

默认：

$$\lambda_g = 0.25, \qquad \lambda_q = 1.0. \tag{61}$$

这里有一个实现细节需要注意：代码只对 $\widetilde{r}_i$ 取绝对值，没有对 $\widetilde{g}_i$ 和 $\widetilde{q}_i$ 取绝对值。因此，如果某个测试窗口的 `shift score` 低于训练集平均水平，$\widetilde{q}_i$ 会是负数，会降低最终分数。这并不是数学错误，而是当前实现的设计选择：高于训练常态的分布偏移会加分，低于训练常态的偏移会减分。

7.5 平滑、阈值与最终预测

合成后的训练分数和测试分数会经过 EWAF 平滑：

$$\bar{S}_i = \alpha S_i + (1-\alpha)\bar{S}_{i-1}, \qquad \alpha = \texttt{ewaf-alpha}. \tag{62}$$

默认 $\alpha = 0.3$。

阈值只由训练分数决定：

$$\theta = \max\left(\mathrm{mean}(\bar{\boldsymbol{S}}^{tr}) + k\,\mathrm{std}(\bar{\boldsymbol{S}}^{tr}), \mathrm{Quantile}_p(\bar{\boldsymbol{S}}^{tr})\right), \tag{63}$$

默认 $k = 2.0$、$p = 0.95$。

测试窗口先按阈值得到初始预测：

$$\hat{y}_i = \mathbb{I}(\bar{S}_i^{te} \geq \theta). \tag{64}$$

然后 `apply_min_anomaly_duration` 会删除短于 `min-anomaly-duration` 的异常片段，默认最短持续长度为 50 个窗口。

7.6 训练集统计量在测试集中的作用

训练集估计出的量不是只服务训练，它们会贯穿整个测试流程。

训练集产生的量	代码位置	在测试集中的作用
标准化均值和标准差 μ_n, σ_n	`mra.py:865--867`	测试集用训练集尺度标准化，保证训练和测试处在同一个数值坐标系。
距离先验矩阵	`mra.py:868--873`	若未提供外部物理距离，图先验由训练集数据自动计算，并影响测试时 GCN 专家输出。
`rate_id` 和 `stride`	`mra.py:900`	测试时仍使用训练集推断出的采样率类别和步长，供门控网络和检测器编码使用。
神经网络权重 $\boldsymbol{\theta}$	`mra.py:583, 618--621`	测试补全、检测器重构、专家分歧和 `shift score` 的输入都依赖训练后的权重。
`shift score` 特征均值 $\boldsymbol{\mu}_f$	`mra.py:836`	测试窗口特征要减去训练均值，衡量离训练常态的偏离。
`shift score` 特征标准差 $\boldsymbol{\sigma}_f$	`mra.py:837`	测试窗口特征要除以训练标准差，使不同特征维度可比较。

训练集产生的量	代码位置	在测试集中的作用
各分数通道的训练均值和标准差	`mra.py:740--749`	测试分数通道用训练集尺度归一化，再进入最终分数合成。
异常阈值 θ	`mra.py:1045--1050`	测试分数是否异常完全由训练分数确定的阈值判断。

8 代码实现逐段说明

8.1 build_window_feature_matrix

核心代码逻辑可以概括为：

1. 初始化空列表 `features`；
2. 对四个列切片分别取子窗口；
3. 对每个子窗口沿时间维计算均值和标准差；
4. 对全变量窗口计算相邻差分平均绝对值；
5. 对全变量窗口沿时间维做 `rfft`，取第 1 到第 5 个频点的平均幅值；
6. 沿特征维拼接所有结果，返回 `float32` 矩阵。

从张量角度看，若输入为：

$$\boldsymbol{V} \in \mathbb{R}^{K\times 50\times 18}, \tag{65}$$

那么：

- `block.mean(axis=1)` 把时间维 50 压缩掉；
- `block.std(axis=1)` 同样把时间维压缩掉；
- `np.diff(windows, axis=1)` 把时间维从 50 变为 49，再 `mean(axis=1)` 压缩为 $K \times 18$；
- `np.fft.rfft(windows, axis=1)` 在长度 50 上产生 26 个频点，切片 `[:, 1:6, :]` 取 5 个非零频点，再对这 5 个频点求均值，得到 $K \times 18$。

8.2 compute_distribution_shift_scores

该函数的输入是训练窗口和评估窗口。这里的 `eval_windows` 可以是测试窗口，也可以是其它需要评估的窗口。

函数内部有四个关键变量：

$$\texttt{train_features} \leftrightarrow \boldsymbol{F}^{tr}, \tag{66}$$

$$\texttt{eval_features} \leftrightarrow \boldsymbol{F}^{eval}, \tag{67}$$

$$\texttt{feature_mean} \leftrightarrow \boldsymbol{\mu}_f, \tag{68}$$

$$\texttt{feature_std} \leftrightarrow \boldsymbol{\sigma}_f. \tag{69}$$

它返回两个一维数组：

$$\texttt{train_score} \in \mathbb{R}^{K_{tr}}, \tag{70}$$

$$\texttt{eval_score} \in \mathbb{R}^{K_{eval}}. \tag{71}$$

这两个数组都使用同一组 $\boldsymbol{\mu}_f, \boldsymbol{\sigma}_f$。这保证训练分数和测试分数可比较。

8.3 main 中的接入位置

main 中与 shift score 直接相关的关键语句按顺序是：

1. reconstruct_full_sequence(...) 生成训练集和测试集补全序列；
2. build_standard_windows(train_complete_scaled, zeros) 生成训练补全评分窗口；
3. build_prompt_test_windows(test_complete_scaled, zeros) 生成测试补全评分窗口；
4. compute_distribution_shift_scores(...) 计算训练和测试 shift score；
5. 把结果写入 train_stats["shift_score"] 和 test_stats["shift_score"]；
6. combine_scores(..., shift_weight=args.score_shift_weight) 将其并入最终分数。

这里传入全零掩码的原因是：补全序列已经没有结构性缺失，重新窗口化时不应再把任何位置当作缺失位置。

9 设计思路、优点与局限

9.1 设计思路

shift score 的设计可以概括为一句话：

> 用训练集补全窗口建立一个“正常窗口统计画像”，再看测试补全窗口偏离这张画像多少。

它刻意选择了容易解释的统计量，而不是再训练一个复杂分类器：

- 均值特征看窗口整体水平；
- 标准差特征看窗口内部波动；
- 差分特征看相邻时间变化速度；
- FFT 幅值特征看周期性或振荡强度。

这些特征都能用简单语言解释，因此适合放进异常检测报告里说明。

9.2 为什么和重构误差互补

重构误差依赖模型是否能重建当前窗口。如果模型泛化能力很强，它可能把异常窗口也重构得不错，导致 detector_score 不高。专家分歧也类似，如果两个专家都朝同一个异常模式输出，分歧也可能不明显。

shift score 不直接看模型是否重构成功，而是看补全后的窗口统计形态是否偏离训练集。因此它能补充另一种异常信号：过程整体分布变化。

9.3 当前实现的优点

- 计算简单，不需要额外神经网络；
- 使用训练集统计量，天然符合无监督异常检测设定；
- 特征可解释，便于排查异常来自水平偏移、波动增强、快速跳变还是频域振荡；
- 使用补全窗口，避免原始缺失值直接破坏均值、标准差和 FFT 计算；
- 与原有 `detector_score`、`disagreement_score` 只通过分数合成耦合，改动面较小。

9.4 当前实现的局限

- 特征分组写死为 0:6、6:12、12:18、0:18，对非 18 列或不同变量分组的数据不够通用。
- 只使用每维标准差，不建模特征之间的相关性，因此不是完整马氏距离。
- $\boldsymbol{\mu}_f, \boldsymbol{\sigma}_f$ 没有保存到模型文件中。如果训练和测试分开运行，需要额外持久化。
- `combine_scores` 中没有对 `shift score` 的归一化值取绝对值。低于训练均值的偏移会降低最终分数，这是否符合任务目标需要通过实验确认。
- `shift score` 依赖补全模型。如果补全模型在异常段把异常“抹平”，分布偏移信号也可能被削弱。

10 常见误解与正确理解

容易误解的说法	更准确的理解
`shift score` 是一个训练出来的神经网络分支。	不准确。它没有神经网络层，也没有梯度更新。它只是用训练集窗口特征估计均值和标准差。
`shift score` 直接比较原始训练集和原始测试集。	不准确。它比较的是训练后模型生成的补全窗口，即 `train_complete_scaled` 和 `test_complete_scaled` 重新切成的窗口。
训练集的 `shift score` 越高，说明训练集中有异常。	不一定。训练集分数主要用于建立基准分布和阈值。单个训练窗口分数高只能说明它离训练平均统计形态较远。
测试集会重新估计 `feature_mean` 和 `feature_std`。	不会。测试集只使用训练集估计出的 $\boldsymbol{\mu}_f, \boldsymbol{\sigma}_f$，否则会产生测试信息泄漏。
`shift score` 越低一定越正常。	通常可以这样理解，但最终判断还要看合成分数、EWAF 平滑、阈值和最短异常持续长度。

11 可复现实验与实现建议

11.1 建议输出的诊断量

若后续要进一步研究 `shift score`，建议在实验输出中增加以下诊断量：

- 每个特征维度的 $\mu_{f,j}$ 和 $\sigma_{f,j}$；

- 测试窗口中贡献最大的若干特征维度；
- `shift_score` 在正常段和异常段的均值、标准差、分位数；
- 合成分数前的 $\widetilde{r}$、$\widetilde{g}$、$\widetilde{q}$ 三个通道值；
- 不同 `--score-shift-weight` 下的检测指标变化。

这些诊断量可以帮助判断 `shift score` 是真的捕捉到了异常分布变化，还是只是被补全模型或尺度因素带偏。

11.2 如果要把训练和测试拆开

当前 `compute_distribution_shift_scores` 在同一次调用中同时接收训练窗口和测试窗口，因此 $\boldsymbol{\mu}_f, \boldsymbol{\sigma}_f$ 是局部变量。若未来需要先训练保存、后单独加载测试，应保存以下内容：

- 神经网络权重 $\boldsymbol{\theta}$；
- 标准化器参数 μ_n, σ_n；
- `rate_id` 和 `stride`；
- `shift score` 的 $\boldsymbol{\mu}_f, \boldsymbol{\sigma}_f$；
- 三个分数通道的训练均值和标准差；
- 训练分数阈值 θ。

否则测试时无法复现训练时的分数尺度。

11.3 如果要提升通用性

当前特征构造与 18 列、三组采样率强绑定。若要支持更多数据集，可以考虑：

1. 根据 `rate_id` 自动生成变量组，而不是写死列切片；
2. 把 FFT 频点数写成参数，例如 `--shift-fft-bins`；
3. 对每个特征组输出单独贡献，方便解释；
4. 试验对 $\widetilde{q}$ 取绝对值或只保留正偏移：
$$\max(\widetilde{q}, 0), \tag{72}$$

以避免低于训练均值的分数通道降低总异常分数。

12 结论

`mra.py` 中的 `shift score` 是一个基于训练集补全窗口统计特征的分布偏移分数。它的入口在 `compute_distribution_shift_scores`，主流程接入点在 `main` 的补全序列重新窗口化之后。它没有梯度训练，也不更新神经网络权重；它从训练集补全窗口中估计特征均值和标准差，再用这些统计量衡量测试补全窗口的偏离程度。

从设计上看，`shift score` 与 `detector_score` 和 `disagreement_score` 互补。前者看补全后的窗口统计形态是否偏离训练分布，后两者分别看检测器重构误差和专家不一致程度。三者经过训练集尺度归一化后合成最终分数，再通过 EWAF、训练阈值和最短异常持续长度得到最终预测。

因此，理解 `shift score` 时应抓住三点：第一，它的输入是训练后模型补全出的稠密窗口；第二，它的“训练参数”是训练窗口特征均值和标准差；第三，它在测试集中通过标准化距离和最终分数合成发挥作用，而不是通过神经网络前向层直接输出异常标签。